Wellness Hub

Integrated Digital Health Monitoring Platform

Document	Project Report
Platform	Flutter Mobile App with Express.js and MongoDB Backend
Submission Deadline	01 May 2026
Prepared By	[Group Name / Student Names]
Repository	[GitHub Link]

1. Executive Summary

Wellness Hub is a mobile health monitoring application designed to help users manage daily wellness activities, personal health logs, goals, reminders, vitals, hydration, sleep, analytics, and profile information in one integrated platform. The project provides a practical digital solution for personal health tracking by combining a Flutter mobile frontend with a RESTful Express.js backend and MongoDB database.

The application focuses on usability and automation. Users can register and log in securely, view daily health rings, track steps where device sensors are available, quickly add water intake, start and stop sleep tracking, record activities, update goals, log vitals, and review analytics. The system is designed as a full-stack application with token-based authentication and persistent user data.

2. Problem Statement

Many users track health information in separate apps, notes, or manual records. This makes it difficult to view overall wellness progress and identify trends. Users also need fast workflows for daily tasks such as logging water intake, sleep, exercise, weight, and health readings. The project addresses this by providing a single mobile application that organizes personal health information and presents it through dashboards and analytics.

3. Aim and Objectives

Aim

To develop an integrated digital health monitoring platform that allows users to record, monitor, and analyze personal wellness data through a mobile application.

Objectives

- Provide secure user registration, login, token refresh, logout, and password reset support.
- Allow users to manage their profile, height, weight, age, gender, and BMI-related information.
- Support daily activity tracking with duration, steps, distance, calories, and notes.
- Enable health logs for weight, height, BMI, date, and notes.
- Enable vitals and wellness logging including blood pressure, heart rate, blood glucose, oxygen, temperature, hydration, sleep, mood, and pain.
- Provide goal creation, goal progress updates, reminders, tips, settings, and analytics.
- Present daily progress through ring-style dashboard metrics and analytic charts.
- Store user data in a backend database using a structured data model.

4. Scope

The project covers a mobile app, backend API, and MongoDB database. It is intended for individual wellness monitoring and academic demonstration. It is not a replacement for professional medical advice or clinical diagnosis.

5. System Features

Module	Description
Authentication	Register, login, logout, refresh token, password reset.
Dashboard	Daily rings for movement, steps, sleep, and water, plus quick tracking controls.
Activities	Add and delete exercise/activity records with calories, distance, steps, and duration.
Goals	Create measurable goals and update progress until completion.
Health Logs	Record weight, height, BMI, date, and notes with validation.
Vitals & Wellness	Record blood pressure, heart rate, glucose, oxygen, temperature, hydration, sleep, mood, and pain.
Analytics	Charts and trend views for activities, vitals, hydration, sleep, and goals.
Reminders	Create health routine reminders and toggle them on or off.
Settings	Manage units, daily targets, privacy preferences, connected-app placeholders, export, and logout.
Automation	Device step counting when supported, estimated calories, quick water logging, and sleep timer.

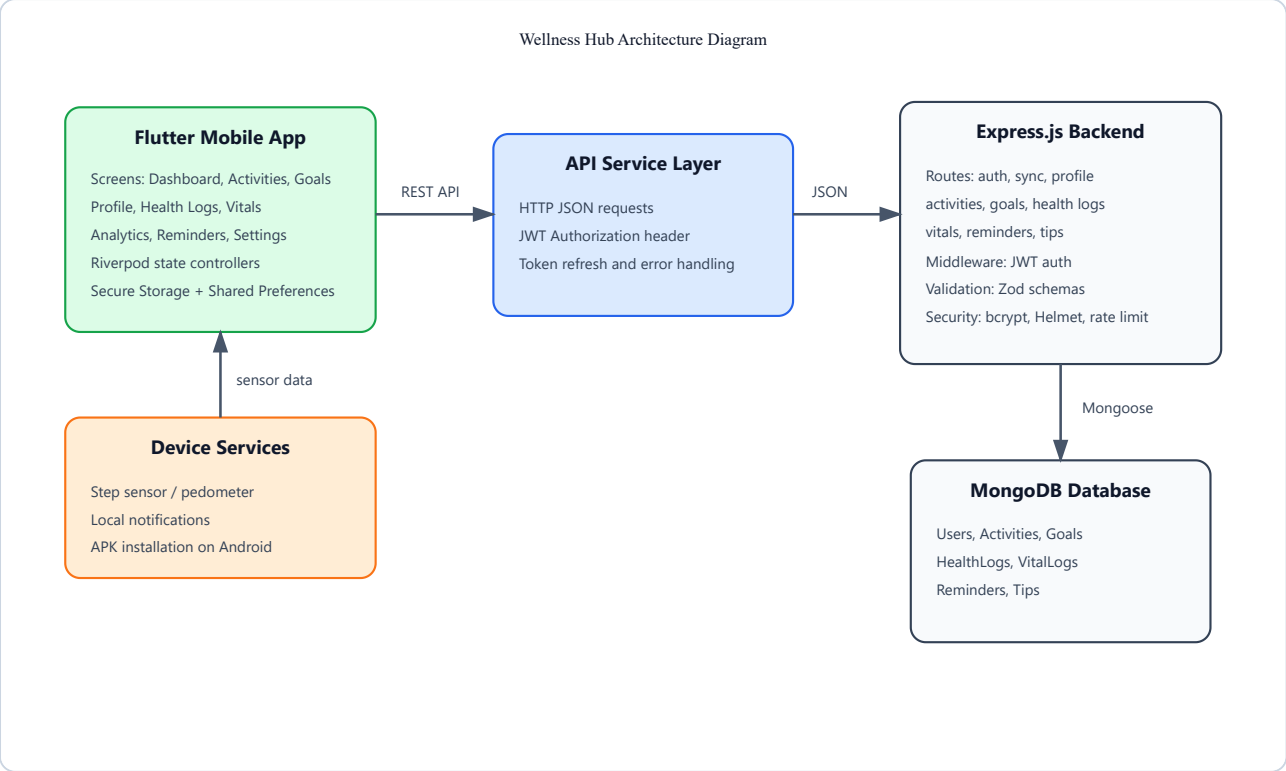
6. Technologies Used

Layer	Technology	Purpose
Frontend	Flutter, Dart	Cross-platform mobile UI and app logic.
State Management	Riverpod	Application state, user data, automation state.
Routing	GoRouter	Navigation and route protection.
Charts	fl_chart	Analytics graphs and trends.
Storage	Shared Preferences, Secure Storage	Local preferences and auth tokens.
Backend	Node.js, Express.js	REST API and business logic.

Database	MongoDB, Mongoose	Persistent data storage and schemas.
Security	JWT, bcryptjs, Helmet, Rate Limiting	Authentication and backend protection.
Mobile Services	Pedometer, Local Notifications	Step counting and reminders.

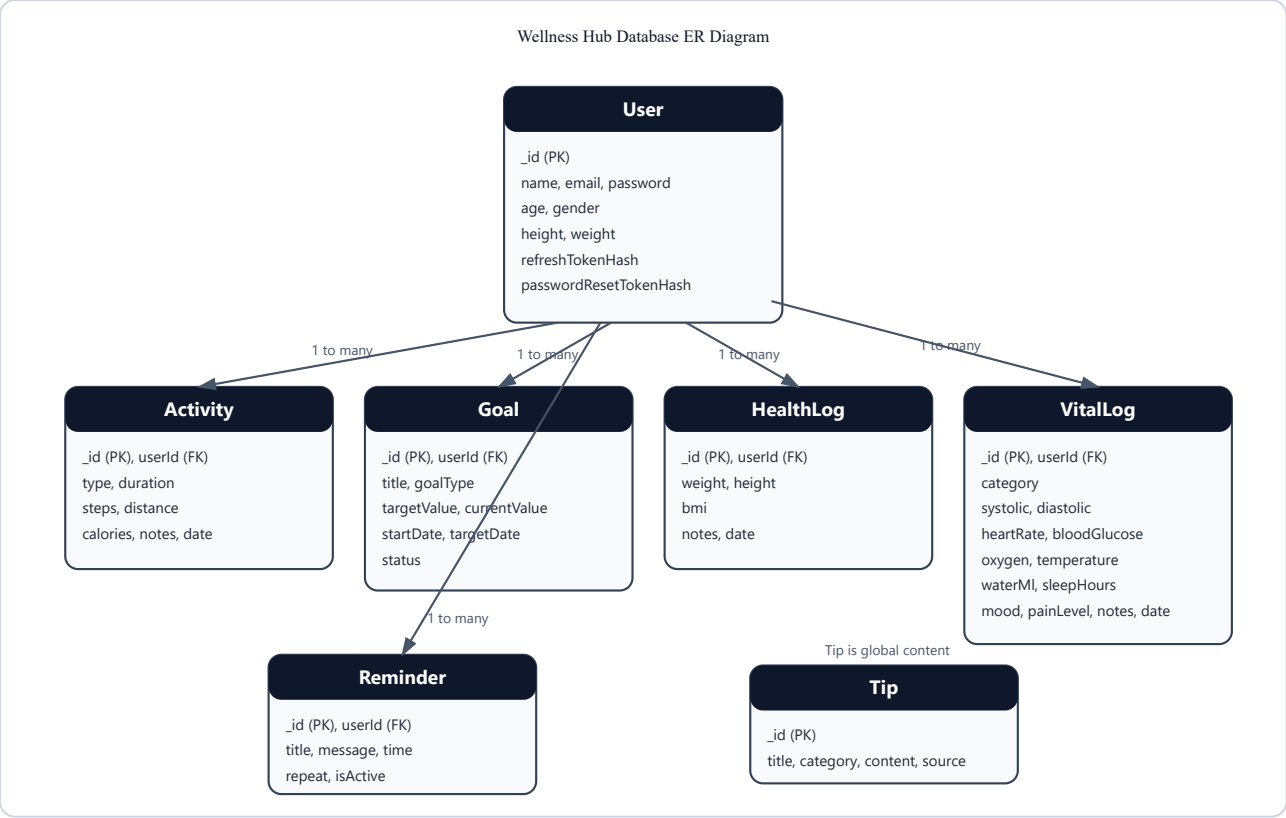
7. System Architecture

The architecture follows a client-server model. The Flutter app communicates with the backend through REST API requests. The backend validates requests, authenticates protected routes using JWT middleware, and stores or retrieves records through Mongoose models connected to MongoDB.



8. Database Design

The database uses separate collections for users, activities, goals, health logs, reminders, tips, and vital logs. User-owned collections reference the user through `userId`. This keeps each user's health data isolated while allowing the backend to load a complete synchronized state after login.



Main Entities

Entity	Important Attributes
User	name, email, password, age, gender, height, weight, refresh token fields, password reset fields
Activity	userId, type, duration, steps, distance, calories, notes, date
Goal	userId, title, goalType, targetValue, currentValue, startDate, targetDate, status
HealthLog	userId, weight, height, bmi, notes, date
VitalLog	userId, category, blood pressure fields, heart rate, glucose, oxygen, temperature, water, sleep, mood, pain, notes, date
Reminder	userId, title, message, scheduledTime, repeat, isActive

Tip	title, category, summary, content, source

9. Implementation Overview

Frontend

The frontend is organized by feature modules under `lib/features`. Shared UI components, form options, health metrics, and chart/ring widgets are stored under `lib/shared`. Data classes are defined in `lib/models/entities.dart`. Backend communication is handled by `ApiService`, while Riverpod controllers manage wellness state and automation state.

Backend

The backend is organized into routes, models, middleware, and utilities. Routes expose endpoints for authentication, profile, activities, goals, health logs, reminders, vitals, tips, and synchronization. Mongoose models define database schemas, and Zod validation protects input data before writing to the database.

10. Security and Validation

- Passwords are hashed with bcrypt before storage.
- JWT access tokens protect private API routes.
- Refresh tokens are hashed before storage.
- Password reset tokens are hashed and expire after a fixed time.
- Zod schemas validate API request payloads.
- Secure storage is used for mobile authentication tokens.
- Helmet and rate limiting are included in the backend for safer HTTP behavior.

11. Testing and Verification

Test Area	Verification	Status
Flutter static analysis	`flutter analyze`	**Passed**
Android build	`flutter build apk`	**Built APK**
Backend syntax	Node.js syntax checks and server run	**Verified**
Authentication	Register, login, token storage, logout	**Implemented**
Data modules	Activities, goals, logs, vitals, reminders	**Implemented**
Mobile emulator	Handled unsupported step sensor gracefully	**Fixed**

12. Limitations

- Step counting depends on device sensor support and permissions.
- Clinical integrations such as hospital systems or certified medical devices are not included.
- Health platform sync with Apple Health or Google Fit is represented as a future integration.
- The app is for wellness tracking and academic demonstration, not diagnosis.

13. Future Enhancements

- Connect Google Fit and Apple Health for richer automatic tracking.
- Add push notifications through a cloud messaging service.
- Add role-based care-provider dashboards.
- Add CSV/PDF export for user health records.
- Add advanced abnormal-vital alerts and trend recommendations.

14. Conclusion

Wellness Hub successfully demonstrates a full-stack mobile health monitoring platform. It combines a usable Flutter interface, REST API, MongoDB persistence, authentication, analytics, automation, and structured health records. The system meets the core academic requirements for a database-backed mobile application and provides a strong foundation for future health technology enhancements.